A Software-SPI Linux Character Device Driver for the MAX7219 Eight-Digit Display

Hadi Alkhub
School of Information and Communication Technology
Griffith University, Queensland, Australia
Student ID: s5331680
e-mail: hadi.alkhub@griffithuni.edu.au

***Abstract*—This paper presents the architectural design, kernel implementation, and empirical validation of a Linux character device driver for an eight-digit, seven-segment MAX7219 display interfaced with a Raspberry Pi 4. The hardware is exposed to user space through the Virtual File System (VFS) as `/dev/max_screen`. Communication uses a software bit-banged Serial Peripheral Interface (SPI) Mode 0 protocol via the Linux GPIO consumer descriptor framework. The driver implements standard file operations (`open`, `release`, `write`, `llseek`) and custom `ioctl` controls for blanking, digital intensity, and cursor visibility. Reentrancy and state encapsulation are maintained with a per-open shadow buffer in `file->private_data`. Hardware cursor tracking is achieved by setting the decimal-point bit (`0x80`). Two user-space programs demonstrate the interface: a sequential automated test suite and an interactive, non-blocking digital clock with brightness stepping and a marquee alarm. Design alternatives—hardware SPI, static driver buffers, in-band control escapes, and per-function `switch` maps instead of `get_val`—are evaluated and rejected. A root-cause analysis covers overloading of `f_pos` in `my_read`.**

***Index Terms*—Linux device drivers, character device, MAX7219, bit-banged SPI Mode 0, GPIO consumer API, VFS, ioctl, Raspberry Pi 4.**

I. Introduction

Device drivers form the abstraction layer in Unix-like operating systems that isolates raw hardware from user-space policy. Under Linux, peripherals appear as files in `/dev`, so `open()`, `read()`, `write()`, `lseek()`, and `ioctl()` are the uniform application interface.

This project develops an out-of-tree Linux kernel module that presents an 8-digit, 7-segment LED display driven by the Maxim MAX7219 [2] as a character device on a Raspberry Pi 4. The laboratory brief imposes the following constraints [1]:

1) Communication must use software-generated (bit-banged) SPI Mode 0 over GPIO rather than the Raspberry Pi hardware SPI master.
2) The current cursor position must be shown on the hardware by illuminating that digit's decimal point.
3) Brightness and cursor decimal-point toggling must use `ioctl`, not escape codes in the write stream.

A central course principle is the separation of *mechanism* from *policy*. The driver provides mechanism only: GPIO bit serialization, spatial coordinate inversion, cursor tracking, and intensity register updates. User-space programs define policy: clock formatting, 1 Hz flashing, keybindings, and alarm timing.

The remainder of this report is organized as follows. Section II reviews the MAX7219 and the Linux character-device model. Section III outlines the layered architecture and GPIO allocation. Section IV justifies the software SPI implementation. Section V details the kernel module. Section VI describes the user-space applications. Section VII is a post-mortem of the `read` path and IOCTL diagnostics. Section VIII evaluates outcomes against the rubric, and Section IX concludes.

II. Background and Theoretical Model

A. MAX7219 Hardware Operating Model

The Maxim MAX7219 is a common-cathode display multiplexer for up to eight 7-segment digits [2]. It contains an on-chip BCD Code-B decoder, an 8×8 dual-port static RAM, multiplex scan circuitry, and a 16-step digital pulse-width modulation (PWM) intensity controller.

Data transfer uses 16-bit serial packets, most-significant bit (MSB) first:

$$\text{Packet}_{16} = (\text{Address}_8 \ll 8) \mid \text{Data}_8. \tag{1}$$

Bits D_{15}–D_{12} are not relevant. Bits D_{11}–D_8 select the internal register ($0x0$–$0xF$), and bits D_7–D_0 are the payload. Active-low chip select (`LOAD`/`CS`) must stay low during transmission and rise with or after the 16th rising clock edge to latch the command. Clock idle-low with data sampled on the rising edge is SPI Mode 0.

Table I lists the registers used by the driver.

TABLE I
MAX7219 essential register address map [2].

Register	Addr.	Description
No-Op	`0x00`	Cascading / shift-register padding
Digits 1–8	`0x01`–`0x08`	Digit RAM (1 RHS … 8 LHS)
Decode Mode	`0x09`	`0xFF`: Code-B on all digits
Intensity	`0x0A`	PWM from 1/32 (`0x00`) to 31/32 (`0x0F`)
Scan Limit	`0x0B`	`0x07`: all eight digits
Shutdown	`0x0C`	`0x01` normal, `0x00` shutdown
Display Test	`0x0F`	`0x01` all LEDs on, `0x00` normal

With Code-B enabled (register `0x09` = `0xFF`), the font ROM maps a 4-bit nibble to 0–9, dash (`0x0A`), `E` (`0x0B`),

H (0x0C), L (0x0D), P (0x0E), and blank (0x0F). Bit 7 ($D_7 = 0x80$) lights the decimal point independently of glyph decode.

B. Linux Kernel Device Driver Architecture

Modern Linux drivers are typically loadable kernel modules. A fully static kernel that bundled every peripheral would waste memory; `insmod` claims resources only when the hardware is present.

Character drivers reach user space through the VFS via `struct file_operations` in `<linux/fs.h>`. C99 designated initializers leave unused callbacks `NULL`:

```
static struct file_operations fops = {
    .owner          = THIS_MODULE,
    .open           = my_open,
    .release        = my_release,
    .read           = my_read,
    .write          = my_write,
    .unlocked_ioctl = my_ioctl,
    .llseek         = my_llseek,
};
```

Pointers must not be dereferenced across the user/kernel boundary. Transfers use helpers from `<linux/uaccess.h>`:

- `copy_from_user()` / `get_user()`: import from user space; uncopied bytes or `-EFAULT` on bad addresses.
- `copy_to_user()` / `put_user()`: export to user space.

Errors are negative `errno` values from `<linux/errno.h>`:

- `-EFAULT`: unmapped address during a copy.
- `-EINVAL`: invalid parameter (e.g., bad seek origin).
- `-ENOMEM`: allocator failure.
- `-ENOTTY`: unknown or inappropriate `ioctl`.

III. System Architecture

VFS abstraction, register logic, and GPIO toggling are split across source files. Fig. 1 shows the layers.

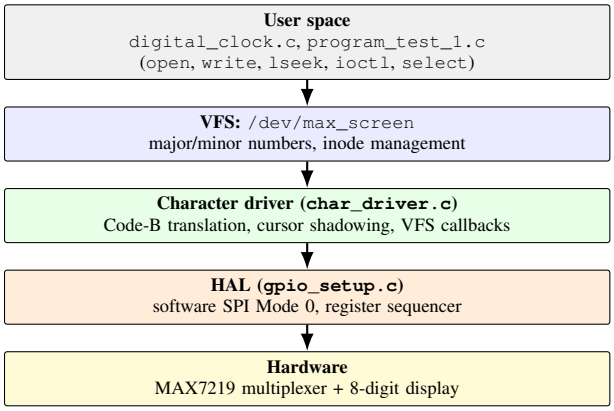


Fig. 1. Layered architecture: applications, VFS operations, and bit-banged SPI.

The source tree is:

- `src/char_driver.c`: module init, device-number allocation, sysfs class, and VFS callbacks.
- `src/gpio_setup.c`: GPIO descriptors, SPI Mode 0 bit-banging, and display setup.
- `src/gpio_setup.h`: prototypes for the SPI helpers.
- `Makefile`: Kbuild `obj-m` multi-part build, load, node permissions, and user programs.

IV. Hardware Interfacing and Bit-Banged SPI

A. Software SPI Justification

The Raspberry Pi 4 exposes a hardware SPI master through `spi-bcm2835`. Hardware SPI was rejected for two reasons:

1) **Specification:** the brief forbids the kernel SPI subsystem and requires synthesizing the protocol on GPIO [1].
2) **Pin flexibility:** bit-banging can use arbitrary GPIOs without pinmux contention on the dedicated SPI0 pins.

B. Pin Mapping and the GPIO Consumer API

Older modules used integer GPIO numbers from `<linux/gpio.h>`. Current kernels prefer the descriptor consumer API in `<linux/gpio/consumer.h>`. On 64-bit Debian kernels for the BCM2711, the GPIO chip base is typically 512, so BCM index n is

$$\text{global descriptor index} = 512 + n. \tag{2}$$

Table II lists the mapping. BCM GPIOs 26, 19, and 13 sit on header pins 37, 35, and 33, which shortens wiring and leaves SPI0 (GPIO 8–11) unused.

TABLE II
Hardware Interface Pin Connections.

Signal	Role	BCM	Offset
`DIN`	Serial data in	26	538
`CLK`	Serial clock	19	531
`CS`	Chip select	13	525
`VCC`	5 V supply	Pin 2	—
`GND`	Ground	Pin 39	—

C. Bit-Banging Mechanics and Timing

The MAX7219 allows f_{CLK} up to 10 MHz (100 ns period). Minimum clock high/low times t_{CH}, t_{CL} are 50 ns; data setup t_{DS} is 25 ns [2]. Serialization in `gpio_setup.c` is:

```
void screen_send_bits(unsigned char address,
                      unsigned char data)
{
    int bits_to_send = (address << 8) | data;
    gpiod_set_value(cs_gpio13, 0);

    for (int i = 0; i <= 15; i++) {
        int curr_bit =
            (bits_to_send >> (15 - i)) & 1;
        gpiod_set_value(din_gpio26, curr_bit);
        gpiod_set_value(clk_gpio19, 1); /* latch */
        gpiod_set_value(clk_gpio19, 0); /* idle */
    }
    gpiod_set_value(cs_gpio13, 1); /* execute */
}
```

Prototype code under `gpio test/` used `udelay(1)`; the production path does not insert extra delays.

D. Coordinate Orientation

On typical 8-digit MAX7219 modules, digit 1 (register `0x01`) is the rightmost element and digit 8 (`0x08`) is the leftmost [2]. Unix strings index 0 on the left. The driver maps

$$\text{hardware address} = 8 - \text{cursor position}. \tag{3}$$

At curr $= 0$ the byte goes to register 8 (leftmost digit), matching left-to-right text.

V. Kernel Module Design and Implementation

A. Module Lifecycle and Sysfs

Registration in `screen_init()` uses dynamic allocation:

1) `alloc_chrdev_region()` reserves a major number and minor 0.
2) `cdev_init()` binds the `cdev` to `fops`.
3) `cdev_add()` registers the instance.
4) `class_create("my_class")` and `device_create()` publish `/sys/class/my_class/max_screen`. `udevd` then creates `/dev/max_screen`.

On `screen_exit()`, the driver shuts the display down (`0x0C`, `0x00`), removes sysfs nodes, deletes the `cdev`, unregisters numbers, and idles the GPIOs.

B. Display Initialization and Lamp Test

`start_display()` programs known register values:

- Decode (`0x09` ← `0xFF`): Code-B on all digits.
- Intensity (`0x0A` ← `0x0F`): maximum PWM.
- Scan limit (`0x0B` ← `0x07`): eight digits.
- Shutdown (`0x0C` ← `0x01`): normal operation.
- Display test (`0x0F` ← `0x00`): test mode off.

`my_open()` writes `0x0F`,`0x01` (all segments on), sleeps five seconds with `msleep(5000)`, then restores `0x0F`,`0x00`, meeting the lamp-test requirement [1].

C. State Management and Shadow Buffering

`my_open()` allocates an 8-byte kernel shadow:

```
char *buff = kzalloc(8 * sizeof(char), GFP_KERNEL);
file->private_data = buff;
file->f_pos = 0;
```

The kernel `struct file` provides `private_data` specifically so a driver can attach per-open device state rather than a global buffer [3]. Storing the shadow there keeps concurrent opens isolated. `kfree()` runs in `my_release()`.

D. Code-B Translation (`get_val`)

The decoder accepts a small character set [2]. A `switch` (or long `if` chain) in `my_write()`, `my_llseek()`, and `my_ioctl()` would duplicate the same map and drift out of sync. Translation is therefore factored into one helper:

```
static char get_val(char c)
{
    if (c >= '0' && c <= '9') return c - '0';
    if (c == 'E' || c == 'e') return 0x0B;
    if (c == 'H' || c == 'h') return 0x0C;
    if (c == 'L' || c == 'l') return 0x0D;
    if (c == 'P' || c == 'p') return 0x0E;
    if (c == '-')             return 0x0A;
    return 0x0F; /* blank */
}
```

ASCII digits are contiguous: `c - '0'` yields Code-B $0x00$–$0x09$.

E. Write Semantics and Wrapping

`my_write()` `kmalloc`s a staging buffer and `copy_from_user`s the payload. For each character:

1) `\n` and `\r` are skipped so `echo` works cleanly.
2) The glyph is sent to register $8 - \text{curr}$.
3) ASCII is stored in `buffer[curr]`.
4) `curr` advances and wraps from 7 to 0.

After the loop, if cursor tracking is on, `0x80` is OR-ed onto the resting digit.

F. File Repositioning (`my_llseek`)

`SEEK_SET`, `SEEK_CUR`, and `SEEK_END` are supported. On a seek:

1) The old digit is rewritten without `0x80`, clearing its decimal point.
2) The new offset is computed from `whence` and clamped to $[0, 7]$.
3) The glyph at the new index is resent, with `0x80` if the cursor is visible.
4) `filep->f_pos` is updated and returned.

This encodes a single-cursor model: at most one decimal point, tied to `f_pos`.

G. The IOCTL Interface

Commands use magic number `'k'`:

- `SCREEN_CTL1_DISABLE` (`_IO`): blank all digits with Code-B `0x0F`, clear the shadow, reset the cursor to 0.
- `SCREEN_CTL2_WRITE` (`_IOW`): `__get_user` an intensity byte, clamp to `0x0F`, store it, write register `0x0A`.
- `SCREEN_CTL3_READ` (`_IOR`): `__put_user` the saved intensity.
- `SCREEN_CTL4_TOGGLE` (`_IOW`): set cursor-DP visibility and refresh register $8 - \text{curr}$.

H. Seconds Decimal-Point Blink

The clock blinks the rightmost digit by seeking to file position 7 and passing $\text{tm_sec} \bmod 2$ into `SCREEN_CTL4_TOGGLE` on each refresh. Even seconds clear the decimal point; odd seconds set it.

VI. User-Space Applications

A. Automated Validation (`program_test_1.c`)

The test program exercises the interface in order:

1) **IOCTL probe:** `SCREEN_CTL3_READ` for default intensity.
2) **Write:** numeric strings and Code-B translation.
3) **Seek:** `SEEK_SET`, `SEEK_CUR`, `SEEK_END` and DP relocation.
4) **PWM:** intensity `0x01` and `0x0F`.
5) **Blank:** `SCREEN_CTL1_DISABLE` on exit.

B. Interactive Digital Clock (`digital_clock.c`)

The demonstration clock formats `localtime()` as `HH-MM-SS` (12-hour) or `DD-MM-YY` (calendar).

1) Non-Blocking Event Loop: A 1 Hz-style refresh must not block on `fgets()`. `select()` watches stdin with a 100 ms timeout:

```
FD_ZERO(&read_fds);
FD_SET(0, &read_fds);
halt.tv_sec  = 0;
halt.tv_usec = 100000; /* 100 ms */

if (select(1, &read_fds, NULL, NULL, &halt) > 0) {
    if (fgets(fgString, sizeof(fgString),
              stdin) != NULL) {
        /* parse command */
    }
}
```

2) Flashing Seconds Indicator: The brief asks for a one-second flash on the rightmost digit [1]. The clock `lseeks` to position 7 and calls `ioctl(..., SCREEN_CTL4_TOGGLE, &dot)` with tm_sec mod 2.

3) Marquee Alarm: `alarm X` sets $t_{\text{alarm}} = t_{\text{now}} + X$. When the deadline is reached, an 8-character window slides across a padded dash string (" --- " with leading/trailing spaces) every 200 ms via `lseek(0)` and `write()`. `select()` still watches stdin; `#` clears the alarm and returns to the clock.

4) Brightness and Exit: Keys > and < step PWM across all levels. Escape (ASCII 27) sends `SCREEN_CTL1_DISABLE`, closes the descriptor, and exits.

VII. Diagnostic Analysis and Implementation Issues

A. Root-Cause Analysis of `my_read`

The assignment requires `read()` to return characters from the leftmost digit up to the cursor [1]. The shadow buffer holds those characters, but `my_read()` returns no useful data.

```
if (*fpos > 0)
    return 0; /* treated as EOF */

if (count > filep->f_pos)
    count = filep->f_pos;
if (count == 0)
    return 0;
```

Listing 1. Flawed length and EOF checks in `my_read`.

The defect is **overloading `file->f_pos` as both a circular cursor and a sequential file offset**:

1) After `open`, `f_pos`=0, so `count` is clamped to 0 and the call returns empty.
2) After a write, `f_pos`> 0, but `if(*fpos > 0) return 0;` treats the cursor as end-of-file.
3) Assigning `*fpos = ret` then corrupts the display cursor.

1) Architectural Resolution: Separate the circular cursor from the stream offset in a per-open struct:

```
struct max_screen_dev {
    char shadow[8];
    unsigned int cursor_pos;
    unsigned int read_offset;
};
```

`cursor_pos` tracks the decimal point; `read_offset` tracks VFS consumption.

VIII. Evaluation and Experimental Validation

The suite was run on a Raspberry Pi 4 Model B. Table III maps features to status.

TABLE III
Rubric specification compliance.

Feature	Mechanism	Status
Open lamp test	`my_open`: `0x0F`, `0x01`, 5 s	Pass
Close clear	`my_release`: Code-B blank 1–8	Pass
SPI Mode 0	GPIO consumer bit-bang	Pass
Cursor track	DP bit `0x80`	Pass
Circular write	Code-B map, 8 − curr	Pass
Seek	`SET`/`CUR`/`END`	Pass
IOCTL	Blank, intensity, DP toggle	Pass
Clock	12-hour + flashing seconds	Pass
Marquee alarm	Sliding window, '#' cancel	Pass
`read()`	LHS through cursor	**Fail**

IX. Conclusion and Future Work

This paper presented a Linux character device driver and user-space suite for a MAX7219 8-digit display on a Raspberry Pi 4. Bit-banged SPI Mode 0 on GPIO maps the protocol onto standard Unix file operations and keeps mechanism separate from policy.

Development required Code-B refactoring, a `select()`-based clock loop, and IOCTL direction alignment. The `my_read()` post-mortem shows that circular cursor state must not share `f_pos` with sequential reads. Future work includes the in-kernel SPI master with Device Tree overlays and hardware-timed PWM [2].

References

[1] Advanced Computer Systems, "Device driver project: weeks 4–7," School of Information and Communication Technology, Griffith University, Course Specification, 2026.

[2] Maxim Integrated, *MAX7219/MAX7221 Serially Interfaced, 8-Digit LED Display Drivers*, Datasheet 19-4452, Rev. 4, 1996.

[3] Linux Kernel Labs, "Character device drivers," *The Linux Kernel* documentation. [Online]. Available: https://linux-kernel-labs.github.io/refs/heads/master/labs/device_drivers.html

Appendix A
Build, Deployment, and Verification

```
# Compile module and user programs
make
make digital_clock
make program_test_1

# Load the driver
sudo insmod max_screen_driver.ko

# Logs and node
dmesg | tail -n 15
ls -l /dev/max_screen
cat /proc/devices | grep max_screen

# Tests
./program_test_1
./digital_clock

# Unload
sudo rmmod max_screen_driver
make clean
```

APPENDIX B
PROPOSED STRUCTURE-BASED MY_READ

This sketch separates stream offset from the display cursor (Section VII-A):

```
struct max_screen_dev {
    char shadow_buf[8];
    unsigned int cursor_pos;
    unsigned int read_offset;
};

static ssize_t robust_read(struct file *filep,
        char __user *user_buf, size_t count,
        loff_t *fpos)
{
    struct max_screen_dev *dev =
        filep->private_data;
    size_t available, to_copy;

    if (dev->read_offset >= dev->cursor_pos)
        return 0;

    available = dev->cursor_pos - dev->read_offset;
    to_copy = min(count, available);

    if (copy_to_user(user_buf,
            dev->shadow_buf + dev->read_offset,
            to_copy))
        return -EFAULT;

    dev->read_offset += to_copy;
    return to_copy;
}
```